

Seismic-Induced Atom Interferometer Phase Noise

Technical Report for the Desktop Calculator
Implementation

Prepared from the local `python` software

Yiming MENG

Date: July 7, 2026



Contents

Executive Summary	1
1 Computation Overview	2
1.1 Processing chain	2
1.2 Input data and preprocessing	3
2 Model Equations	4
2.1 From acceleration ASD to displacement ASD	4
2.2 From displacement ASD to laser phase ASD	4
2.3 Atom interferometer sensitivity function	4
2.4 Cycle-time response term	5
2.5 Atom phase PSD and ASD	5
3 Numerical Integration and RMS Estimation	6
3.1 Cumulative RMS curve	6
3.2 Band-limited RMS reported to the user	6
4 Parameter Definitions	8
5 Exported Results	9
6 Implementation Notes and Limitations	10
Appendix: Relation to the Source Code	11

List of Figures

1.1	Computation chain implemented in the desktop application.	3
-----	---	---

List of Tables

4.1	User-adjustable parameters in the desktop application	8
5.1	Meaning of exported result columns	9

Executive Summary

Purpose and Scope

This report documents the computation implemented in the desktop application that estimates how measured ground acceleration noise contributes to atom interferometer phase noise. The program accepts a frequency column and one acceleration amplitude spectral density (ASD) column from an Excel worksheet, converts the acceleration ASD into displacement ASD, transforms displacement into laser phase ASD, filters that phase noise with the atom interferometer sensitivity function and an additional cycle-time response term, and finally integrates the resulting atom phase power spectral density (PSD) to produce an RMS phase fluctuation in radians.

The emphasis of this report is implementation fidelity rather than a broad theoretical review. Every equation and parameter described below is matched to the current code path in `app_desktop.py`, including the preprocessing rules, the numerical trapezoidal integration, the default parameter values, and the meaning of the exported result columns.

Chapter 1

Computation Overview

1.1 Processing chain

Implemented workflow

The application starts from a measured acceleration ASD as a function of frequency. The code interprets the selected acceleration column as a one-sided ASD with units of acceleration per square-root bandwidth. Each frequency sample is converted into angular frequency, the acceleration ASD is divided by ω^2 to obtain displacement ASD, and the displacement ASD is multiplied by an optical phase conversion factor to obtain laser phase ASD. The laser phase ASD is then squared and weighted by the atom interferometer sensitivity function and by a cycle-time response term. The final PSD is integrated over frequency to obtain the variance, and the square root of that variance is reported as the RMS atom phase noise.

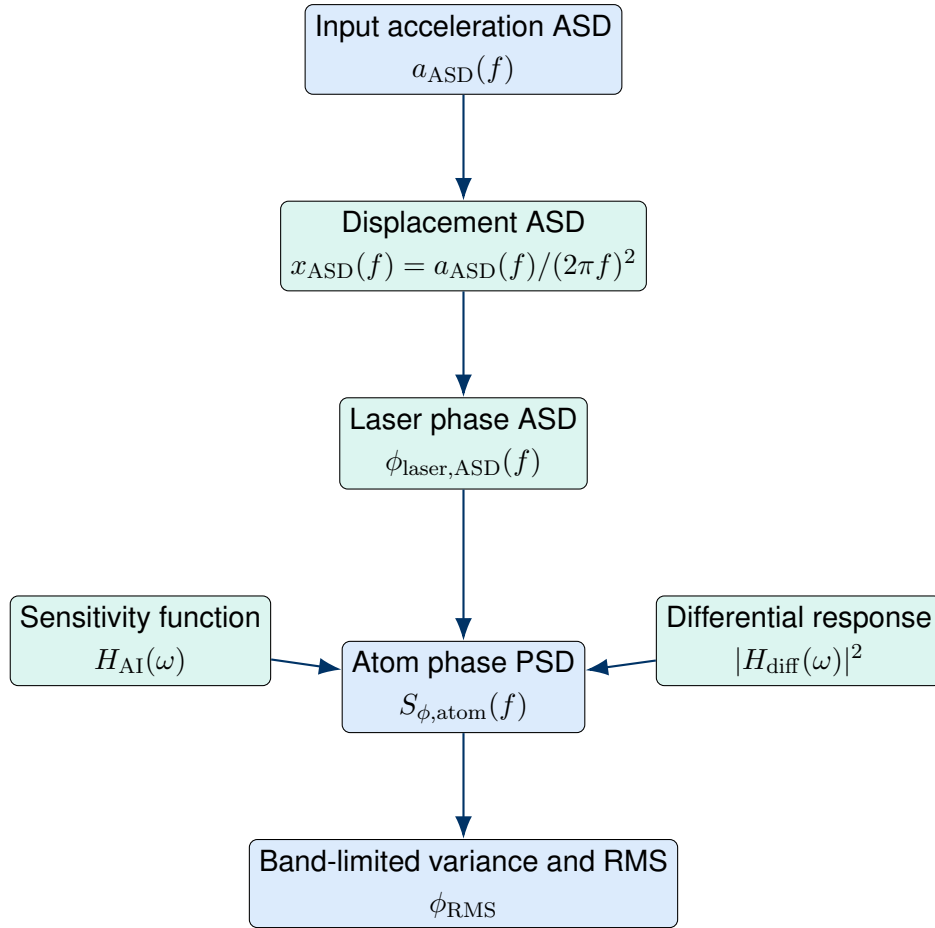


Figure 1.1: Computation chain implemented in the desktop application.

1.2 Input data and preprocessing

The reader should note that the program is designed to be permissive about spreadsheet formatting. The helper function `parse_numeric_series()` accepts ordinary floating-point values, scientific notation with either E or D, decimal commas used in French-format spreadsheets, embedded spaces, non-breaking spaces, and strings that carry additional unit text. After parsing, the application drops missing and non-finite values, removes all samples with non-positive frequency or negative ASD, sorts the data in ascending frequency, and removes duplicate frequency values. This preprocessing is important because the later spectral integration assumes a strictly ordered frequency axis with at least two valid samples.

If the user leaves the integration bounds blank, the application uses the minimum and maximum frequency present in the cleaned data. If the user supplies explicit bounds, the code verifies that $f_{\min} < f_{\max}$ and that at least two cleaned samples fall inside the requested band before performing the final integration.

Chapter 2

Model Equations

2.1 From acceleration ASD to displacement ASD

The first physical step is the frequency-domain double integration of acceleration into displacement. With f in hertz and $\omega = 2\pi f$ in radians per second, the code computes

$$\omega = 2\pi f, \quad (2.1)$$

followed by

$$x_{\text{ASD}}(f) = \frac{a_{\text{ASD}}(f)}{\omega^2} = \frac{a_{\text{ASD}}(f)}{(2\pi f)^2}. \quad (2.2)$$

This relation is consistent with rigid-body kinematics in the frequency domain. If the acceleration ASD is expressed in $\text{m s}^{-2} \text{Hz}^{-1/2}$, the resulting displacement ASD is expressed in $\text{m Hz}^{-1/2}$.

2.2 From displacement ASD to laser phase ASD

The next step converts the mirror or platform displacement ASD into laser phase ASD through a linear scale factor,

$$\phi_{\text{laser,ASD}}(f) = \left(\frac{R\pi}{\lambda} \right) x_{\text{ASD}}(f), \quad (2.3)$$

where λ is the laser wavelength and R is a dimensionless phase coefficient. In the default configuration of the application, $\lambda = 780 \text{ nm}$ and $R = 4.0$. The code keeps R user-adjustable because the appropriate scale factor depends on the optical geometry used in the interferometer model.

2.3 Atom interferometer sensitivity function

The desktop application implements the following sensitivity function for the interferometer,

$$H_{\text{AI}}(\omega) = 8 \sin\left(\frac{\omega T}{2}\right) \sin^2\left(\frac{\omega T}{4}\right), \quad (2.4)$$

where T is the interrogation time. In the code, the default value is $T = 0.4 \text{ s}$. The squared magnitude H_{AI}^2 is used later when constructing the phase PSD.

2.4 Cycle-time response term

To describe the additional response associated with the experimental cycling time, the code multiplies by

$$|H_{\text{diff}}(\omega)|^2 = C \sin^2 \left(\frac{\omega T_c}{2} \right), \quad (2.5)$$

where T_c is the cycling time between consecutive experimental cycles and C is a dimensionless prefactor. The default values used by the application are $T_c = 1.0$ s and $C = 2.0$. Because C and T_c are exposed as user inputs, the program can be adapted to different experimental repetition rates without changing the source code.

2.5 Atom phase PSD and ASD

The final atom phase PSD is built by multiplying the laser phase ASD squared by the two response factors,

$$S_{\phi, \text{atom}}(f) = |H_{\text{diff}}(\omega)|^2 H_{\text{AI}}^2(\omega) \phi_{\text{laser, ASD}}^2(f). \quad (2.6)$$

The code then forms an atom phase ASD for plotting purposes through

$$\phi_{\text{atom, ASD}}(f) = \sqrt{\max(S_{\phi, \text{atom}}(f), 0)}. \quad (2.7)$$

The $\max(\cdot, 0)$ safeguard is a numerical protection against tiny negative values that may appear through floating-point roundoff; it does not change the model itself.

Chapter 3

Numerical Integration and RMS Estimation

3.1 Cumulative RMS curve

The cumulative RMS shown in the last plot tab is built incrementally with a trapezoidal sum over the discrete frequency grid. For consecutive samples (f_i, S_i) and (f_{i+1}, S_{i+1}) , the code computes the segment variance contribution

$$\Delta\sigma_{\phi,i}^2 = \frac{1}{2} (S_{i+1} + S_i) (f_{i+1} - f_i), \quad (3.1)$$

and then accumulates

$$\sigma_{\phi,\text{cum}}^2(f_n) = \sum_{i=1}^{n-1} \Delta\sigma_{\phi,i}^2, \quad (3.2)$$

so that the displayed cumulative RMS is

$$\phi_{\text{RMS,cum}}(f_n) = \sqrt{\max(\sigma_{\phi,\text{cum}}^2(f_n), 0)}. \quad (3.3)$$

This output is useful when one wants to identify which frequency bands dominate the total phase noise budget.

3.2 Band-limited RMS reported to the user

The scalar RMS displayed in the control panel corresponds to a trapezoidal integration over the selected frequency band,

$$\sigma_{\phi}^2[f_{\min}, f_{\max}] = \int_{f_{\min}}^{f_{\max}} S_{\phi,\text{atom}}(f) df, \quad (3.4)$$

implemented numerically as

$$\sigma_{\phi}^2[f_{\min}, f_{\max}] \approx \text{trapz}(S_{\phi,\text{atom}}(f), f), \quad (3.5)$$

with the final RMS reported as

$$\phi_{\text{RMS}} = \sqrt{\max(\sigma_{\phi}^2, 0)}. \quad (3.6)$$

The application displays the RMS in radians, the corresponding variance in radians squared, and the frequency band used for the integration.

Chapter 4

Parameter Definitions

Table 4.1 summarizes the user-adjustable model parameters. The key practical point is that λ , T , T_c , R , and C are model parameters, whereas $f_{\min}$ and $f_{\max}$ only affect the integration band used to summarize the already computed PSD.

Table 4.1: User-adjustable parameters in the desktop application

Parameter	Default	Units	Meaning in the code
λ	780	nm	Laser wavelength used in the displacement-to-phase conversion of Eq. (2.3).
T	0.4	s	Interrogation time appearing in the atom interferometer sensitivity function of Eq. (2.4).
T_c	1.0	s	Cycling time between consecutive experimental cycles appearing in the response factor of Eq. (2.5).
R	4.0	dimensionless	Phase conversion coefficient that scales displacement ASD into laser phase ASD.
C	2.0	dimensionless	Prefactor that scales the cycle-time response term $ H_{\text{diff}} ^2$.
$f_{\min}$	data minimum	Hz	Lower frequency bound used only in the final variance integration.
$f_{\max}$	data maximum	Hz	Upper frequency bound used only in the final variance integration.

The parameters R and C deserve special care because they are effective coefficients rather than fundamental constants. Their physically correct values depend on the optical path, the phase readout convention, and the exact way in which two interferometer channels are differenced. The defaults in the application are reasonable starting values, but they should be verified against the user's instrument model before quantitative conclusions are drawn.

Chapter 5

Exported Results

The CSV export is not limited to the final RMS value. Instead, the application writes the intermediate quantities needed to audit the calculation. Table 5.1 lists the main exported columns and their interpretation.

Table 5.1: Meaning of exported result columns

Column name	Units	Interpretation
frequency_hz	Hz	Cleaned frequency grid used for all subsequent operations.
acceleration_asd	$\text{m s}^{-2} \text{Hz}^{-1/2}$	Parsed acceleration ASD taken from the selected worksheet column.
omega_rad_s	rad/s	Angular frequency $\omega = 2\pi f$.
displacement_asd_m_sqrtHz	$\text{m Hz}^{-1/2}$	Displacement ASD obtained from Eq. (2.2).
laser_phase_asd_rad_sqrtHz	$\text{rad Hz}^{-1/2}$	Laser phase ASD obtained from Eq. (2.3).
H_AI	dimensionless	Atom interferometer sensitivity function evaluated on the frequency grid.
H_diff_squared	dimensionless	Cycle-time response term of Eq. (2.5). The legacy column name is retained for compatibility.
atom_phase_psd_rad2_Hz	$\text{rad}^2 \text{Hz}^{-1}$	Final atom phase PSD obtained from Eq. (2.6).
atom_phase_asd_rad_sqrtHz	$\text{rad Hz}^{-1/2}$	Square root of the PSD after clipping tiny negative roundoff.
cumulative_rms_rad	rad	Running RMS obtained by cumulative trapezoidal integration.

Chapter 6

Implementation Notes and Limitations

Current model scope

The application is intentionally compact and deterministic. It assumes that the selected spreadsheet column already represents a valid acceleration ASD as a function of frequency and that the displacement-to-phase coupling is fully captured by a single coefficient R . It also assumes that the full interferometer response can be represented by the specific sensitivity function in Eq. (2.4) and by the sinusoidal cycle-time response term in Eq. (2.5). No cross-axis coupling, resonant mechanical transfer function, aliasing model, or additional readout noise term is introduced in the present implementation.

In practice, these assumptions are often adequate for a first-order vibration-noise estimate or for comparing several measured vibration spectra under a common instrument model. They should not, however, be mistaken for a full end-to-end simulation of the instrument. If the user later needs a more realistic model, the natural extensions are to insert a measured mechanical transfer function before Eq. (2.3), replace R by a geometry-specific wave-vector model, and add other independent phase-noise PSD contributions before the final integration.

Appendix: Relation to the Source Code

The equations in this report map directly onto the following helper functions and variables in the desktop application: `sensitivity_function()` implements Eq. (2.4); `cycle_response_squared()` implements Eq. (2.5); `integrate_trapezoid()` performs the band-limited spectral integration; the `calculate()` method constructs the intermediate arrays and result columns; and `clean_data()` enforces the data-quality checks described earlier. This direct mapping should make it straightforward to maintain the report alongside future code updates.